

rest_ext: A Generic REST API Extension for DuckDB

Executive Summary

rest_ext is a DuckDB extension that lets any REST API be queried with plain SQL. A user ATTACHes an API the same way they would ATTACH a Postgres or MySQL database, and every endpoint becomes a callable table function returning rows whose schema is inferred automatically from the API's own JSON responses. No connectors, no per-API code, no ETL scripts - configuration alone is enough to turn a REST API into a SQL-queryable data source.

Problem

DuckDB already has first-class scanners for relational databases (postgres_scanner, mysql_scanner) and for flat files (read_json, read_csv, read_parquet). It has no equivalent for the REST APIs that a large share of real-world "cost and usage" data actually lives behind - cloud billing APIs, SaaS admin consoles, internal microservices. Today, getting that data into DuckDB means writing and maintaining a bespoke script per API: handling auth, pagination-adjacent request shaping, and flattening JSON into rows by hand.

The concrete motivating case: pulling Google Workspace Admin SDK cost and usage reports into DuckDB for analysis. That API - like most REST APIs - needs an authenticated request to a JSON-returning endpoint. Nothing about that is Google-specific; the same shape covers GCP billing exports, Stripe, GitHub, or any internal service with an OpenAPI spec.

Proposed Solution

rest_ext treats a REST API as an attachable database:

```
INSTALL rest_ext;  
LOAD rest_ext;  
ATTACH '<config>' AS myapi (TYPE rest_ext, ...);  
SELECT * FROM myapi.<endpoint>(query_params, body);
```

Three ways to configure an attachment, in increasing order of power:

- Single endpoint - just a URL, headers, and method. Produces one bare callable.
- Namespace - a hand-written map of name -> URL. Produces one callable per name, dot-qualified (myapi.Users(...), myapi.Orders(...)).
- OpenAPI spec-import - point ATTACH at an OpenAPI 2.0 or 3.x document (a local file or a remote SPEC_URL), and every operation the spec defines becomes a callable automatically, already wired to the right URL, method, and path.

In every mode, the response schema is not fixed up front - it is inferred from the actual JSON returned, at query-bind time. A JSON object becomes a STRUCT column, an array becomes a LIST column, and scalars

map to native DuckDB types. Path parameters (`/pets/{petId}`) are filled in from the same query-params object a caller already passes, with no separate syntax to learn.

Authentication is handled generically, independent of any one provider's scheme: headers can be supplied inline, or stored once via `CREATE SECRET` and matched to a target URL automatically, exactly like DuckDB's existing S3/https secrets.

Architecture

- Storage extension + custom Catalog - ATTACH is backed by DuckDB's `StorageExtension` interface (the same mechanism `postgres_scanner` uses), not a one-off scalar function, so attachments behave like real databases: DETACH cleans up fully, and namespace mode gets a real Catalog/Schema per attachment.
- Dynamic schema inference - the HTTP call happens at bind time so the JSON response shape can determine output columns before DuckDB needs them, rather than requiring a schema to be declared up front.
- Generic HTTP layer - one shared request path (built on `cpp-httplib` with TLS) handles headers, query parameters, JSON bodies, and `{parameter}` path-template substitution for every mode and every method (GET/POST/PUT/PATCH/DELETE).
- OpenAPI import - parses both the modern `servers-array` style (3.x, including server variables and relative server URLs) and the legacy `host/basePath/schemes` style (2.0), so either generation of spec works unmodified.
- Secrets integration - a dedicated `rest_ext_headers` secret type, redacted in `duckdb_secrets()`, resolved per-request by longest-scope-prefix match against the target URL.

Demonstration: the Swagger Petstore API

No local file needed - `SPEC_URL` fetches the spec directly from the live API:

```
ATTACH 'petstore' AS petstore (TYPE rest_ext, FORMAT 'openapi',  
    SPEC_URL 'https://petstore3.swagger.io/api/v3/openapi.json');
```

Every operation in the spec (`findPetsByStatus`, `getPetById`, `addPet`, ...) is now a callable, already pointed at the right URL and method:

```
SELECT id, name, status  
FROM petstore.findPetsByStatus('{"status":"available"}', '{}')  
LIMIT 5;
```

Result:

id	name	status
1790355770971	Rex-1790355770971	available
1790355837460	Rex-1790355837460	available
1790355839043	Rex-1790355839043	available
1790356921263	Rex	available
1790356930581	Rex-1790356930581	available

A path parameter (/pet/{petId}) is filled in from the same query-params object, with no separate placeholder syntax:

```
SELECT id, name, status
FROM petstore.getPetById( '{"petId":"1790356921263"}', '{}');
```

Result:

id	name	status
1790356921263	Rex	available

Nothing about findPetsByStatus or getPetById was hand-written for this demo - both came directly out of parsing the spec.

Applying This: Google Workspace Admin Console Usage Reports

This is the original motivating case: pulling usage data - and, by extension, the license counts that drive per-seat cost - out of the Google Workspace Admin Console, without writing a Google-specific integration. Google publishes this as the Admin SDK Reports API, a real, documented REST API (base URL <https://admin.googleapis.com>), with no spec-import needed - two endpoints, configured by hand in namespace mode, cover the whole reporting surface:

Report	Method & Path	Key Parameters
Customer usage	GET /admin/reports/v1/usage/dates/{date}	customerId, parameters, maxResults
User usage	GET /admin/reports/v1/usage/users/{userKey}/dates/{date}	customerId, parameters, filters

parameters is a comma-separated list of app:field names. The customer usage report's accounts app exposes exactly the license/quota figures that drive Workspace's per-seat billing - accounts:apps_total_licenses, accounts:apps_used_licenses, accounts:used_quota_in_mb - the closest real, documented proxy this API offers to "cost" (a true dollar figure is a Cloud Billing / BigQuery-export concern, one layer up from Workspace itself; license consumption is what Workspace usage reporting actually tracks).

Configuring both endpoints is the same namespace-mode pattern used everywhere else in this proposal, with

a bearer token stored once as a secret scoped to Google's host:

```
CREATE SECRET (  
  TYPE rest_ext_headers,  
  HEADERS MAP {'Authorization': 'Bearer <oauth2-access-token>'},  
  SCOPE 'https://admin.googleapis.com'  
);  
  
ATTACH 'resources={  
  "CustomerUsage": "https://admin.googleapis.com/admin/reports/v1/usage/dates/{date}",  
  "UserUsage": "https://admin.googleapis.com/admin/reports/v1/usage/users/{userKey}/dates/{date}"  
}' AS gadmin (TYPE rest_ext);  
  
SELECT *  
FROM gadmin.CustomerUsage(  
  '{"date": "2026-09-01", "customerId": "my_customer",  
    "parameters": "accounts:apps_total_licenses,accounts:apps_used_licenses,accounts:used_quota_in_mb"}',  
  '{}'  
);
```

This was run against Google's real, live endpoint (with a placeholder bearer token, not a valid one) to confirm the request actually reaches it correctly:

```
IO Error: rest_ext: HTTP request to  
"https://admin.googleapis.com/admin/reports/v1/usage/dates/2026-09-01" returned status 401:  
{ "error": { "code": 401, "message": "Request had invalid authentication credentials. ...",  
  "status": "UNAUTHENTICATED" }}
```

The {date} path parameter was substituted correctly, and Google's own server answered with its real, documented 401 response for a missing/invalid token - not a connection failure or a client-side parsing error. A valid OAuth 2 access token (scope <https://www.googleapis.com/auth/admin.reports.usage.readonly>) is the only missing piece for this to return real usage rows.

Status and Validation

The extension has been validated end-to-end against three classes of API, not just synthetic fixtures:

- Long-stable public test data (jsonplaceholder.typicode.com) - the primary regression suite, covering single-endpoint mode, namespace mode, dynamic schema inference for nested JSON, secrets (redaction, override precedence, per-host resolution), and OpenAPI spec-import mechanics (operationId naming, path parameters, server-variable substitution, error paths).
- The live OpenAPI 3.x Swagger Petstore - validates spec-import against a real, unmodified, third-party spec, including a relative servers URL (a real, standard OpenAPI pattern the spec author used, resolved against wherever the spec itself was fetched from).
- The live Swagger 2.0 ("legacy") Petstore - validates the older host/basePath/schemes base-URL convention against a real spec, exercising the one code path the jsonplaceholder-based suite alone never touched.

These live-API checks run as a separate, explicitly-invoked test suite (.test_slow), kept apart from the default regression suite since they depend on outbound network access and a third party's uptime and data.

Next Steps

- Parse OpenAPI security schemes directly from a spec (API key / bearer / basic), so a spec's own declared auth requirement can suggest the right CREATE SECRET shape instead of the caller having to already know it.
- Apply the extension to the original motivating case - Google Workspace Admin SDK reports - as the first real, non-demo integration, using OpenAPI spec-import against Google's own published discovery documents where available.
- Investigate pagination conventions (next_page_token, Link headers) as a generic, opt-in layer on top of the existing per-call model, rather than a provider-specific special case.